

1/3rd term Programming Fundamentals

Oct. 2 2026, 18:30-21:30h

- You can solve the problems in any order. Solutions must be submitted to the automated judgement system Themis. For each problem, Themis will test ten different inputs, and check whether the outputs are correct.
- Grading: You get one grade point for free. The remaining nine points are based solely on the judgement given by Themis. The first problem is worth one grade point. The remaining four problems are worth two grade points each.
- You can score partial points if a program passes several but not all test cases. For example, if you pass 6 out of 10 test cases, then you are awarded 60% of the grade points for that exercise.
- Inefficient programs may be rejected by Themis. In such cases, the error will be 'time limit exceeded'. The time limit for each problem is one second.
- The number of submissions to Themis is not limited. No points are subtracted for multiple submissions.
- There will be no assessment of programming style. However, accepted solutions are checked manually for cheating: For example, precomputed answers will not be accepted, even though Themis accepts them.
- Needless to say, you are not allowed to work together. If plagiarism is detected, both parties (supplier of the code and the person that sends in copied code) will be excluded from any further participation in the course.
- You are not allowed to use email, phones, smartwatches, tablets, calculators, etc. There is a calculator available on the exam computers (see icon on the desktop). You are allowed to consult the ANSI C book and a dictionary. You are not allowed to use a printed copy of the reader or the lecture slides, however they are available digitally (as a pdf) in Themis. You are allowed to access your own submissions previously made to Themis for the lab sessions (other submissions are not visible).
- For each problem, the first three test cases (input files) are available on Themis. These input files, and the corresponding output files, are called `1.in`, `2.in`, `3.in`, `1.out`, `2.out` and `3.out`. These files can be used to test whether the output of your program matches the requested layout, so that there can be no misunderstanding about the layout and spaces in the output.
- **If you fail to pass a specific test case of a problem, then you are advised not to lose much time on debugging your program, and continue with another problem. In the last hour of the exam, all test cases will be made visible in Themis.**

Problem 1: Clock

A digital clock displays time in 24-hour format. A time is given by two integers h and m , where h is the hour and m is the number of minutes after that hour. Write a program that computes the time after d minutes have passed. The clock wraps around after midnight. For example, 25 minutes after 23:50 is 00:15.

The input consists of three integers h , m , and d , where $0 \leq h < 24$, $0 \leq m < 60$, and $0 \leq d \leq 10000$.

The output must be the resulting time in the format HH:MM. Both the hour and the minutes must always be printed using two digits, including a leading zero when necessary.

Example 1:

input:

14 45 80

output:

16:05

Example 2:

input:

23 50 25

output:

00:15

Example 3:

input:

8 30 30

output:

09:00

Problem 2: Road to zero

Consider the following operation on a positive integer. First, reverse the order of its digits. Then replace the number by the absolute difference between the original number and the reversed number.

For example, starting with 123, the reversed number is 321, so the new number is

$$|123 - 321| = 198.$$

Repeating the operation gives

$$123 \rightarrow 198 \rightarrow 693 \rightarrow 297 \rightarrow 495 \rightarrow 99 \rightarrow 0.$$

Thus, starting with 123, it takes 6 steps to reach 0. Note that leading zeroes in a reversed number are ignored. For example, the reverse of 120 is 21.

The input for this problem consists of a single integer n , where $1 \leq n \leq 10^6$. Write a program that determines how many steps are required to reach 0 for the given input number n .

For some numbers, the process does not reach 0. However, the inputs used in the test cases are guaranteed to be such that the process eventually reaches 0.

The output must be a single integer: the number of steps required to reach 0.

Example 1:

input:

123

output:

6

Example 2:

input:

101

output:

1

Example 3:

input:

100

output:

2

Problem 3: Unit Fractions

Some fractions with numerator 1 can be written as the sum of two other fractions that also have numerator 1. For example,

$$\frac{1}{4} = \frac{1}{5} + \frac{1}{20} = \frac{1}{6} + \frac{1}{12} = \frac{1}{8} + \frac{1}{8}.$$

Thus, there are three different ways to represent $\frac{1}{4}$ as the sum of two such fractions.

Write a program that, given input n , determines the number of pairs of positive integers (a, b) satisfying

$$\frac{1}{n} = \frac{1}{a} + \frac{1}{b}$$

Since exchanging a and b gives the same representation, only solutions for which $a \leq b$ are counted. The input consists of a single integer n , where $2 \leq n \leq 10000$. The output must be a single integer: the number of different pairs (a, b) satisfying $\frac{1}{n} = \frac{1}{a} + \frac{1}{b}$.

Example 1:

input:

4

output:

3

Example 2:

input:

6

output:

5

Example 3:

input:

7

output:

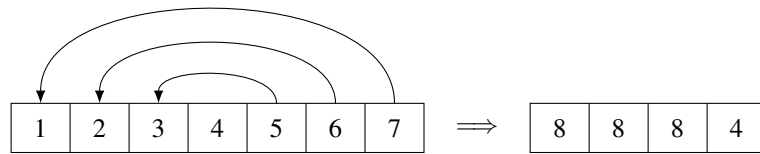
2

Problem 4: Array Folding

Consider an array of integers written from left to right. The array can be folded by folding its right half over its left half. The rightmost element is placed on top of the leftmost element, the second element from the right is placed on top of the second element from the left, and so on. The values of elements that are placed on top of each other are added.

If the array contains an odd number of elements, the middle element is not covered and remains unchanged. For example, folding the array $[1, 2, 3, 4, 5, 6, 7]$ once gives $[8, 8, 8, 4]$, and folding it a second time gives $[12, 16]$ (see figure below).

First fold

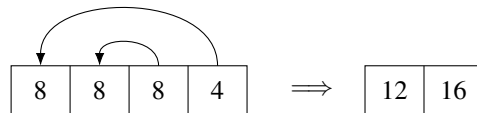


The right half is folded over the left half.

The middle element is not covered.

$$1 + 7 = 8, \quad 2 + 6 = 8, \quad 3 + 5 = 8$$

Second fold



$$8 + 4 = 12, \quad 8 + 8 = 16$$

Write a program that performs k folds on a given array and outputs the resulting array. The first line of the input contains two integers n and k , where n is the number of elements in the array and k is the number of folds to perform, with $1 \leq n \leq 1000$ and $1 \leq k \leq 10$. The second line contains an array with n integers, each between 0 and 10000 (inclusive). The test cases are chosen such that the array contains at least two elements before each fold.

The output must be the resulting array after k folds. The elements must be enclosed in square brackets and separated by commas, without spaces.

Example 1:

input:

7 2
1 2 3 4 5 6 7

output:

[12, 16]

Example 2:

input:

6 1
1 2 3 4 5 6

output:

[7, 7, 7]

Example 3:

input:

5 1
1 2 3 4 5

output:

[6, 6, 3]

Problem 5: Designated Survivor

Consider an array of positive integers arranged from left to right. Initially, the first element is the current element. Its value determines how far to jump through the array.

If the current element has value k , jump k elements to the right. If a jump passes the last element of the array, it wraps around to the first element and continues from there. After the jump, remove the element from which the jump started. The element where the jump ends becomes the new current element.

For example, consider

$$[3, 1, 4, 2, 5].$$

Initially, the current element is 3. Jumping three elements to the right ends at 2. The 3 is then removed, and 2 becomes the current element:

$$[3, 1, 4, 2, 5] \longrightarrow [1, 4, 2, 5].$$

From 2, jumping two elements to the right ends at 1. After removing the 2, the array becomes

$$[1, 4, 5],$$

with 1 as the current element. Continue this process until only one element remains. We call this element the *designated survivor*. For the example, the designated survivor is 5.

Note that it is possible that a jump ends at the same element from which it started. In that case, the element is removed and the element immediately following it becomes the new current element.

Write a program that determines the value of the surviving element. The first line of the input contains an integer n , where $2 \leq n \leq 1000$. The second line contains the array with n positive integers, each between 1 and 10000. The output must be a single integer: the value of the designated survivor.

Example 1:

input:

5

3 1 4 2 5

output:

5

Example 2:

input:

5

1 2 3 4 5

output:

3

Example 3:

input:

5

5 4 3 2 1

output:

1